

Incident vs Problem vs Change

The Real-World Differences



INCIDENT



The car stops suddenly

Goal: Restore service quickly

PROBLEM



The third time in two months

Goal: Identify the root cause and prevent recurrence

CHANGE



Replace the faulty part

Goal: Implement a controlled, approved fix with minimal disruption

MODULE 3: Problem Management & Change Enablement

Introduction: From Firefighting to Prevention and Safe Evolution

Last2x week, we focused on **Incident Management**—the reactive practice of restoring normal service as quickly as possible when things break. Think of Incident Management as the firefighters: their only goal is to put out the fire immediately, even if it means soaking the

carpet or breaking a window to do so.

However, if a fire breaks out in the same server room every single Tuesday, simply putting out the fire over and over again is not enough. Eventually, you must ask: *Why does this room keep catching fire, and how do we prevent it permanently without breaking other systems when we fix it?*

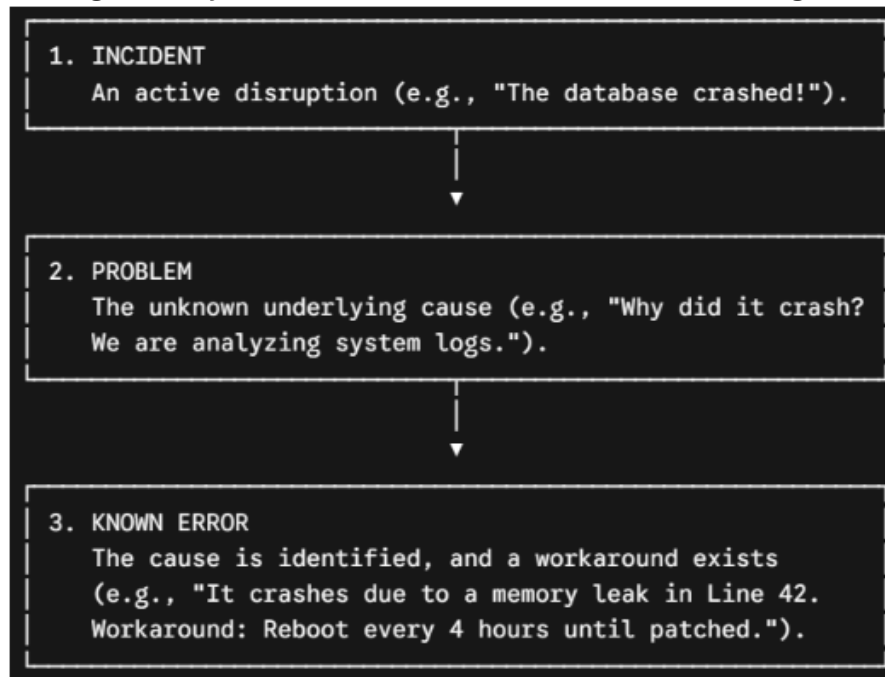
This is where **Problem Management** and **Change Enablement** come in:

- **Problem Management** is *The Detective*: It investigates recurring failures, identifies root causes, and finds permanent solutions.
- **Change Enablement** is *The Safety Inspector*: It ensures that when you deploy fixes, upgrades, or new features, you do so safely without causing *new* outages.

Section 1: Problem Management — Finding the Root Cause

Incident vs. Problem vs. Known Error

To master Problem Management, you must understand the three distinct stages of a system defect:



- **Incident:** Focuses on user impact and speed. (Goal: Restore service now).
- **Problem:** Focuses on understanding why the failure happened. (Goal: Find the root cause).
- **Known Error:** A problem that has been analyzed, its root cause is identified, and a temporary workaround or permanent solution has been documented.

Root Cause Analysis (RCA) Technique: The 5 Whys

One of the simplest yet most effective tools made in Problem Management is the 5 Whys method. You start with the incident and repeatedly ask "Why?" until you strip away surface symptoms and reveal the fundamental flaw.

- **Scenario:** The online campus grading system went down during finals week.
 1. *Why did the system go down?* The database server ran out of memory and froze.
 2. *Why did it run out of memory?* An automated script was executing an unoptimized SQL query repeatedly.
 3. *Why was an unoptimized query running?* A developer pushed new code to production without performance testing.
 4. *Why was it pushed without performance testing?* There was no automated testing pipeline in the deployment process.

5. *Why was there no automated testing pipeline?* The IT department lacked a formal Change Enablement policy for software updates.

Root Cause: Lack of deployment governance and automated testing, **not** just a "bad server."

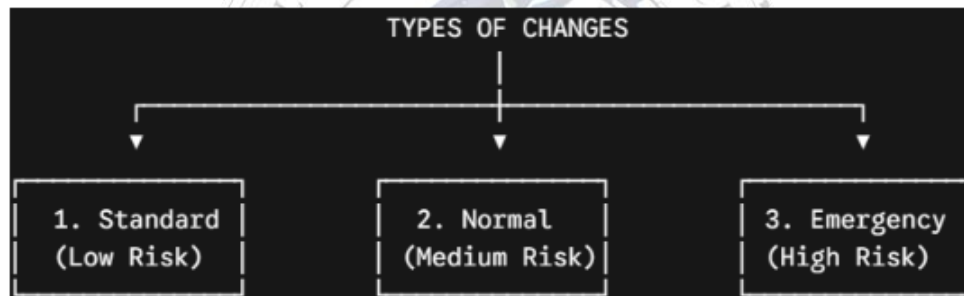
Section 2: Change Enablement — Maximizing Successful Changes

What is Change Enablement?

In ITIL 4, **Change Enablement** (formerly called Change Management) is the practice of ensuring that risks are properly assessed, changes are authorized, and deployments are scheduled safely to minimize service disruptions.

The Golden Rule of Change Enablement: Over 70% of modern IT outages are caused by poorly planned, unauthorized, or untested changes made by internal staff.

The 3 Types of IT Changes



1. Standard Changes:

- **Risk Level:** Low risk, pre-authorized, routine.
- **Process:** Follows an established, well-tested procedure. Does not need approval every time.
- **Examples:** Replacing a failed desktop monitor, resetting a password, or deploying a pre-approved security patch to endpoint laptops.

2. Normal Changes:

- **Risk Level:** Medium-to-high risk; requires formal assessment and authorization.
- **Process:** Must go through a change request, risk assessment, testing in a staging environment, and approval by a **Change Authority** (such as a Change Advisory Board / CAB).
- **Examples:** Upgrading the core database version, migrating email servers to the cloud, or updating the university LMS software.

3. Emergency Changes:

- **Risk Level:** Extremely high urgency; implemented immediately to resolve a critical incident or security threat.
- **Process:** Bypasses standard meeting schedules; approved rapidly by a specialized Emergency Change Authority (ECAB) with retrospective documentation required afterward.
- **Examples:** Patching an active zero-day cybersecurity vulnerability being exploited in real time.

Section 3: Bad Practice in Problem & Change Management

Just like in Incident Management, under pressure, teams often fall into dangerous operational traps:

Feature	Problem Management (RCA)	Change Enablement	Bad Practice (What NOT to do)
Primary Goal	Find the root cause to prevent recurrence.	Deploy updates safely with minimal risk.	Apply "band-aid" fixes indefinitely or push untested code directly to production.
Trigger	Recurring incidents or major system failures.	Planned updates, patches, or problem resolutions.	Fear of deadlines or frustration with formal process approvals.
Operational Consequence	Long-term stability and system health.	Smooth deployments without secondary outages.	Unannounced crashes, undocumented changes, and team conflict.

Example of Bad Practice: "Ninja Deployments"

A developer fixes a bug on their local machine at 4:45 PM on a Friday. Instead of submitting a Change Request and running automated tests, they directly upload the code to the live production server ("Ninja Deployment") and go home for the weekend. At 5:05 PM, the entire system crashes, leaving night-school students unable to access their coursework, with no record of what was changed or who changed it.

Section 4: Self-Reflection & Applied Case Study

Case Study: "The Friday Night Database Upgrade"

Scenario: At 6:00 PM on Friday, an IT specialist notices that the main college registration database is responding slowly. Wanting to be helpful, the technician decides to upgrade the database software version directly on the live server without submitting a Change Request or notifying the Service Desk.

The upgrade fails halfway through, corrupting the database index files. The registrar's office is unable to process student records on Monday morning. Because no one logged the change, the Service Desk spends 6 hours trying to diagnose why the database suddenly stopped working.

Student Journal Questions:

1. **Problem vs. Incident:** Was the initial slow response time at 6:00 PM an *Incident* or a *Problem*? How should it have been handled?
2. **Change Categorization:** Should a database software version upgrade be categorized as a *Standard*, *Normal*, or *Emergency* change? Explain why.
3. **Bad Practice Analysis:** Identify the specific "Bad Practice" committed by the technician. What two steps in the Change Enablement workflow were completely bypassed?
4. **Resolution Strategy:** How would proper integration between Problem Management and Change Enablement have prevented this Monday morning outage?